Documentacao Tecnica — Plataforma Castro Advocacia

Versao: 1.0 (em desenvolvimento) **Data**: Outubro de 2026 **Autor**: Matheus Vitor Carneiro Lira **Repositorio**: `castro-advocacia-chat`

---

1. Visao Geral da Arquitetura

A plataforma e um **monorepo** com dois modulos independentes:

```
castro-advocacia-chat/
├── frontend/    React SPA (cliente)
├── backend/     Spring Boot REST API (servidor)
├── dist/        Build de producao do frontend
├── docs/        (reservado para documentacao)
└── README.md
```

Fluxo de comunicacao:

```
[Navegador] ──HTTP/JSON──> [Spring Boot API :8080] ──JPA──> [PostgreSQL/H2]
     │
     └── SPA React servida por Vite (:5173 em dev)
```

- O frontend faz requisicoes REST ao backend via `fetch`
- Autenticacao por **JWT** (Bearer token no header `Authorization`)
- Sessao stateless — sem estado no servidor

---

2. Stack Tecnologica

Camada	Tecnologia	Versao
Frontend	React	18.3.1
Frontend	TypeScript	5.6.3
Frontend	Vite	5.4.11
Frontend	Tailwind CSS	3.4.17
Frontend	lucide-react (icones)	0.468.0
Backend	Java	21
Backend	Spring Boot	3.4.1
Backend	Spring Security	6.x (via Boot)
Backend	Spring Data JPA	3.x (via Boot)
Backend	springdoc-openapi (Swagger)	2.5.0
Backend	Apache POI (Excel)	5.2.5
Backend	JJWT	0.12.6

Backend	Lombok	via Boot
Banco (dev)	H2 (em memoria)	runtime
Banco (prod)	PostgreSQL	runtime

---

3. Estrutura do Projeto

Backend (`backend/`)

```
src/main/java/com/castroadvocacia/plataforma/
├── PlataformaApplication.java       Classe main
├── config/
│   ├── CorsConfig.java              CORS (localhost:5173, 4173)
│   ├── GlobalExceptionHandler.java  Tratamento global de erros
│   └── SecurityConfig.java          Spring Security + JWT config
├── controller/
│   ├── AuthController.java          POST /api/auth/login, GET /api/auth/me
│   ├── ClienteController.java       CRUD /api/clients
│   ├── DashboardController.java     GET /api/dashboard/stats
│   ├── ImportController.java        POST /api/import, GET/DELETE categorias
│   └── ServicoController.java       CRUD /api/services
├── dto/
│   ├── AuthRequest.java             record(email, senha)
│   ├── AuthResponse.java            record(token, nome, email, perfil)
│   ├── ClienteDTO.java              Cliente + iniciais do nome
│   ├── ClienteRequest.java          Input de criacao/edicao
│   ├── DashboardDTO.java            Estatisticas agregadas
│   ├── ResponsavelDTO.java          Responsavel + iniciais
│   ├── ServicoDTO.java              Servico formatado
│   └── ServicoRequest.java          Input de criacao/edicao
├── model/
│   ├── Cliente.java                 Entidade JPA — tabela `clientes`
│   ├── Processo.java                Entidade JPA — tabela `processos`
│   ├── Servico.java                 Entidade JPA — tabela `servicos`
│   ├── StatusCliente.java           Enum: ATIVO, INATIVO, PENDENTE
│   ├── StatusServico.java           Enum: (nao usado diretamente, status e String)
│   └── Usuario.java                 Entidade JPA — tabela `usuarios`
├── repository/
│   ├── ClienteRepository.java       JpaRepository<Cliente, UUID>
│   ├── ProcessoRepository.java      + findByCategoria, deleteByCategoria
│   ├── ServicoRepository.java       + findByCategoria, listar categorias/statuses
│   └── UsuarioRepository.java       + findByEmail
├── security/
│   ├── CustomUserDetailsService.java  Carrega usuario por email
│   ├── JwtAuthenticationFilter.java   Filtro que valida JWT em cada request
│   ├── JwtUtil.java                   Gera e valida tokens JWT
│   └── RateLimitFilter.java           Rate limit no login (5 tentativas/min)
├── service/
│   ├── ClienteService.java           Logica de negocios de clientes
│   ├── DashboardService.java         Agregacao de estatisticas
```

```
│   ├── ImportService.java            Importacao de planilhas XLSX
│   └── ServicoService.java           Logica de negocios de servicos
└── util/
    └── Iniciais.java                 Extrai iniciais do nome (ex: "MC")
```

Frontend (`frontend/`)

```
src/
├── main.tsx                  Ponto de entrada React
├── App.tsx                   Roteamento hash-based + AuthProvider
├── index.css                 Estilos globais, fonte Lato, CSS vars
├── theme.ts                  Tokens de design (cores, fontes, radii)
├── vite-env.d.ts             Types do Vite
├── assets/                   SVGs (logo, icones)
├── components/
│   ├── clientes/
│   │   └── ClientFormModal.tsx    Modal de criacao/edicao de cliente
│   ├── dashboard/
│   │   ├── StatCard.tsx           Card de KPI (total, ativos, etc.)
│   │   ├── StatusBarChart.tsx     Grafico de barras por status
│   │   └── ExportReportButton.tsx Dropdown de exportacao (PDF/CSV)
│   ├── layout/
│   │   ├── DashboardLayout.tsx    Layout principal (sidebar + conteudo)
│   │   ├── Sidebar.tsx            Menu lateral com navegacao
│   │   └── navItems.tsx           Definicao dos itens do menu
│   ├── pages/
│   │   ├── DashboardPage.tsx      Pagina do painel
│   │   ├── ClientesPage.tsx       Pagina de clientes
│   │   ├── ServicosPage.tsx       Pagina de servicos
│   │   ├── ImportarPage.tsx       Pagina de importacao
│   │   └── LoginPage.tsx          Pagina de login
│   ├── servicos/
│   │   └── ServiceFormModal.tsx   Modal de criacao/edicao de servico
│   └── ui/                        Componentes reutilizaveis
│       ├── Button.tsx
│       ├── Modal.tsx
│       ├── TextField.tsx
│       ├── SelectField.tsx
│       ├── SearchInput.tsx
│       ├── IconButton.tsx
│       ├── ConfirmDialog.tsx
│       ├── PageHeader.tsx
│       ├── StatusTag.tsx
│       └── Avatar.tsx
├── context/
│   └── AuthContext.tsx             Provider de autenticacao
├── hooks/
│   ├── useFetch.ts               Hook generico de fetch com loading/error
│   └── useHashRoute.ts           Roteamento por hash (#dashboard, etc.)
├── lib/
│   ├── api.ts                    Layer centralizada de API (fetch + JWT)
```

```
│   ├── cn.ts                    Utilitario de classes CSS condicionais
│   ├── currency.ts              Formatacao e mascara de moeda (BRL)
│   └── serviceStatus.ts         Mapeamento dinamico de cor por status
└── types/
    └── entities.ts              Tipos TypeScript das entidades
```

---

4. Endpoints da API REST

Base URL: `http://localhost:8080`

Autenticacao

Metodo	Rota	Descricao	Auth
POST	`/api/auth/login`	Login (retorna JWT + dados do usuario)	Nao
GET	`/api/auth/me`	Dados do usuario autenticado	Sim

Request de login:

```
{
  "email": "admin@castroadvocacia.com",
  "senha": "castro2025"
}
```

Response de login:

```
{
  "token": "eyJhbGciOiJIUzI1NiJ9...",
  "nome": "Usuario Adm",
  "email": "admin@castroadvocacia.com",
  "perfil": "ADMIN"
}
```

Clientes

Metodo	Rota	Descricao
GET	`/api/clients`	Listar todos os clientes
GET	`/api/clients/{id}`	Buscar cliente por ID (UUID)
POST	`/api/clients`	Cadastrar novo cliente
PUT	`/api/clients/{id}`	Atualizar cliente
DELETE	`/api/clients/{id}`	Excluir cliente
DELETE	`/api/clients`	Excluir todos os clientes

Body de criacao/edicao:

```
{
  "nome": "Joao da Silva",
  "cpf": "12345678901",
  "numeroProcesso": "0001234-56.2026.8.10.0001",
  "status": "ATIVO"
}
```

Servicos

Metodo	Rota	Descricao
GET	`/api/services`	Listar servicos (opcional: `?categoria=nome`)
GET	`/api/services/categorias`	Listar categorias distintas
GET	`/api/services/statuses`	Listar statuses distintos
GET	`/api/services/{id}`	Buscar servico por ID (UUID)
POST	`/api/services`	Cadastrar novo servico
PUT	`/api/services/{id}`	Atualizar servico
DELETE	`/api/services`	Excluir todos os servicos

Body de criacao/edicao:

```
{
  "titulo": "Processo Administrativo INSS",
  "tipoServico": "Previdenciario",
  "valorTotal": 5000.00,
  "responsavel": "Dr. Castro",
  "status": "Em Atendimento",
  "categoria": "Administrativo"
}
```

Dashboard

Metodo	Rota	Descricao
GET	`/api/dashboard/stats`	Estatisticas agregadas (clientes, servicos, processos)

Importacao de Planilhas

Metodo	Rota	Descricao
POST	`/api/import`	Importar arquivo .xlsx (multipart/form-data, campo `file`)
GET	`/api/import/categorias`	Listar categorias importadas
DELETE	`/api/import/{categoria}`	Remover categoria (processos + servicos associados)

> *Todos os endpoints (exceto login) requerem header* `Authorization: Bearer <token>` .

> *Swagger UI disponivel em `/swagger-ui/index.html` (somente em dev).*

---

5. Autenticacao e Seguranca

JWT (JSON Web Token)

- Algoritmo: HS256
- Expiracao: configuravel via `jwt.expiration` (em milissegundos)
- Secret: configuravel via `jwt.secret`
- Payload: email do usuario + perfil (ADMIN/ADVOGADO/ATENDENTE)
- Token enviado no header: `Authorization: Bearer <token>`
- Armazenado no frontend em `localStorage` (chave: `castro_token`)

Protecao contra forca bruta

O `RateLimitFilter` protege o endpoint de login:

- **5 tentativas** por IP a cada 60 segundos
- Apos exceder: bloqueio de **15 minutos**
- Resposta: HTTP 429 Too Many Requests

Headers de seguranca

Header	Valor
X-Frame-Options	DENY
Strict-Transport-Security	max-age=31536000; includeSubDomains
X-Content-Type-Options	nosniff
Cache-Control	no-cache, no-store

Senhas

- Hash: **BCrypt** (fator padrao do Spring Security)
- Senha padrao do admin: `castro2025` (hash no `data.sql`)

CORS

Origens permitidas (configuravel):

- `http://localhost:5173` (Vite dev)
- `http://localhost:4173` (Vite preview)

---

6. Banco de Dados

Perfis

Perfil	Banco	DDL	Uso
`dev`	H2 em memoria (`jdbc:h2:mem:castrodev`)	`create-drop`	Desenvolvimento local
`prod`	PostgreSQL (via env vars)	`validate`	Producao

Variaveis de ambiente (producao)

Variavel	Descricao	Exemplo
`DATABASE_URL`	URL JDBC do PostgreSQL	`jdbc:postgresql://host:5432/castro`
`DATABASE_USER`	Usuario do banco	`castro_user`
`DATABASE_PASSWORD`	Senha do banco	`****`

Schema das tabelas

`usuarios`

Coluna	Tipo	Restricoes
id	UUID	PK, auto-gerado
nome	VARCHAR	NOT NULL
email	VARCHAR	NOT NULL, UNIQUE
senha	VARCHAR	NOT NULL (BCrypt hash)
perfil	VARCHAR	NOT NULL (ADMIN, ADVOGADO, ATENDENTE)
criado_em	TIMESTAMP	NOT NULL, imutavel

`clientes`

Coluna	Tipo	Restricoes
id	UUID	PK, auto-gerado
nome	VARCHAR	NOT NULL
cpf	VARCHAR(20)	—
numero_processo	VARCHAR	—
status	VARCHAR	NOT NULL (ATIVO, INATIVO, PENDENTE)
criado_em	TIMESTAMP	NOT NULL, imutavel
atualizado_em	TIMESTAMP	auto-atualizado

`servicos`

Coluna	Tipo	Restricoes
id	UUID	PK, auto-gerado
numero	INTEGER	NOT NULL, UNIQUE, auto-incrementado
titulo	VARCHAR	NOT NULL
tipo_servico	VARCHAR	—

valor_total	DECIMAL(12,2)	—
responsavel	VARCHAR	—
status	VARCHAR	NOT NULL (default: "Aberto")
categoria	VARCHAR	—
criado_em	TIMESTAMP	NOT NULL, imutavel
atualizado_em	TIMESTAMP	auto-atualizado

processos

Coluna	Tipo	Restricoes
id	UUID	PK, auto-gerado
categoria	VARCHAR	NOT NULL
corretor	VARCHAR	—
cliente	VARCHAR	—
cpf	VARCHAR	—
status	VARCHAR	—
ano	VARCHAR	—
mes	VARCHAR	—
beneficio	VARCHAR	—
tipo	VARCHAR	—
fisico	VARCHAR	—
protocolado	VARCHAR	—
data_protocolo	VARCHAR	—
protocolo	VARCHAR	—
concessao	VARCHAR	—
aba	VARCHAR	—
tarefas	VARCHAR(2000)	—
observacao	VARCHAR(2000)	—
numero_processo	VARCHAR	—
esp	VARCHAR	—
vara	VARCHAR	—
valor	VARCHAR	—

data_movimentacao	VARCHAR	—
data_implantacao_beneficio	VARCHAR	—
criado_em	TIMESTAMP	NOT NULL, imutavel
atualizado_em	TIMESTAMP	auto-atualizado

Seed Data

O arquivo `data.sql` insere um usuario administrador padrao:

- Nome: `Usuario Adm`
- Email: `admin@castroadvocacia.com`
- Senha: `castro2025` (BCrypt)
- Perfil: `ADMIN`

Observacao sobre migrations

O projeto **nao utiliza** ferramentas de migration (Flyway, Liquibase). O schema e gerado automaticamente pelo Hibernate:

- Em `dev`: recriado a cada reinicio (`create-drop`)
- Em `prod`: apenas validado (`validate`) — o schema deve existir previamente

---

7. Importacao de Planilhas (XLSX)

Funcionamento

O `ImportService` usa **Apache POI** para ler arquivos Excel:

1. Validacao do arquivo (extensao .xlsx, tamanho <= 10MB, assinatura ZIP)
2. Extracao da **categoria** a partir do titulo/nome da planilha
3. Leitura dos headers da primeira linha (mapeamento flexivel, insensivel a acentos)
4. Criacao de registros na tabela `processos` para cada linha
5. Criacao de registros na tabela `servicos` correspondentes
6. Coleta de clientes unicos por CPF

Colunas reconhecidas

O mapeamento e flexivel e aceita variacoes com/sem acentos:

- ANO, MES, CORRETOR, CLIENTE, CPF, STATUS
- BENEFICIO, TIPO, FISICO, PROTOCOLADO, DATA PROTOC.
- PROTOCOLO, CONCESSAO, TAREFAS, OBSERVACAO
- NUMERO DO PROCESSO, ESP, VARA, VALOR
- DATA DA MOVIMENTACAO, DATA DA IMPLANTACAO DO BENEFICIO

---

8. Frontend — Detalhes Tecnicos

Roteamento

Roteamento **hash-based** via hook customizado `useHashRoute`:

- `#dashboard` → DashboardPage
- `#clientes` → ClientesPage
- `#servicos` → ServicosPage
- `#importar` → ImportarPage
- `#login` → LoginPage

Rota padrao: `dashboard`.

Autenticacao no frontend

O `AuthContext` gerencia o estado de autenticacao:

1. No mount, verifica se existe token em `localStorage` (`castro_token`)
2. Se existir, valida com `GET /api/auth/me`
3. Se invalido, remove o token e redireciona para login
4. `login()` envia credenciais, armazena token, e carrega dados do usuario
5. `logout()` remove token e limpa estado

Layer de API (`lib/api.ts`)

Funcao centralizada `request()` que:

- Injeta `Authorization: Bearer <token>` automaticamente
- Redireciona para login em caso de HTTP 401
- Parseia JSON da resposta
- Objetos separados por dominio: `clientsApi`, `servicesApi`, `dashboardApi`, `importApi`

Design System

Tokens extraidos do Figma em `theme.ts`:

- **Cores**: azul (`#5165E1` base), cinza (6 tons), fuschia, status (4 cores), danger
- **Fonte**: Lato (Google Fonts)
- **Radii**: sm (5px), content (20px), shell (24px), full (9999px)
- **Breakpoint**: `lg` (1024px) — alterna layout mobile/desktop

Componentes UI reutilizaveis em `components/ui/`: Button, Modal, TextField, SelectField, SearchInput, IconButton, ConfirmDialog, PageHeader, StatusTag, Avatar.

---

9. Como Rodar Localmente

Pre-requisitos

- **Java 21** (JDK)
- **Node.js 18+** e npm
- **Maven** (ou usar o wrapper `./mvnw` incluso)
- **Git**

Backend

```
cd backend
./mvnw spring-boot:run
```

O servidor inicia em `http://localhost:8080` com perfil `dev` (banco H2 em memoria).

Console H2 disponivel em: `http://localhost:8080/h2-console`

- JDBC URL: `jdbc:h2:mem:castrodev`
- User: `sa` / Senha: (vazio)

Swagger UI: `http://localhost:8080/swagger-ui/index.html`

Frontend

```
cd frontend
npm install
npm run dev
```

O frontend inicia em `http://localhost:5173`.

Build de producao

Frontend:

```
cd frontend
npm run build
```

Gera arquivos estaticos em `dist/`.

Backend:

```
cd backend
./mvnw package -DskipTests
```

Gera JAR executavel em `target/`.

---

10. Deploy em Producao

Variaveis de ambiente necessarias

```
DATABASE_URL=jdbc:postgresql://host:5432/nome_banco
DATABASE_USER=usuario
DATABASE_PASSWORD=senha
SPRING_PROFILES_ACTIVE=prod
```

Checklist de deploy

1. Criar banco PostgreSQL e executar o schema (ou usar `ddl-auto: update` temporariamente)
2. Configurar variaveis de ambiente
3. Executar o JAR: `java -jar plataforma-0.0.1-SNAPSHOT.jar`
4. Servir o frontend (dist/) via Nginx, Caddy ou outro servidor estatico
5. Configurar CORS no backend para o dominio de producao
6. Configurar HTTPS (certificado SSL)
7. Alterar a senha padrao do admin

Consideracoes

- O `jwt.secret` em `application.properties` deve ser substituido por uma variavel de ambiente em producao
- O CORS deve ser atualizado para incluir o dominio de producao
- Recomenda-se usar Flyway ou Liquibase para gerenciar migrations em producao

---

11. Status Atual e Pendencias

Funcionalidades prontas

- Login com JWT e protecao contra forca bruta
- Dashboard com KPIs e graficos dinamicos
- CRUD completo de Clientes
- CRUD completo de Servicos
- Importacao de planilhas XLSX
- Exportacao de relatorios (PDF/CSV)
- Design responsivo (mobile + desktop)
- Swagger UI para documentacao da API

Pendencias conhecidas

- Funcionalidade de alterar/redefinir senha
- Cadastro de novos usuarios pelo painel
- Gerenciamento de perfis/permissoes granulares
- Migrations de banco (Flyway/Liquibase) para producao
- Arquivo `.env.example` com variaveis documentadas
- Testes automatizados (unitarios e integracao)
- Configuracao de CI/CD
- Docker/docker-compose para facilitar deploy
- Integracao com chatbot WhatsApp (n8n + Chatwoot) — em desenvolvimento separado

---

Documento gerado em outubro de 2026. Projeto em desenvolvimento ativo.